

# Rapport Software Engineering

Daboussi Akram

Enseignant : M. Régis Jean-Charles

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>Introduction.....</b>                                               | <b>3</b>  |
| <b>1. Architecture et Choix de conception.....</b>                     | <b>3</b>  |
| 1.1. Le Polymorphisme (Interface BitPacker).....                       | 4         |
| 1.2. Le Pattern 'Factory'.....                                         | 4         |
| 1.3. La Séparation des Responsabilités (Classe Utilitaire).....        | 5         |
| <b>2. Logique d'implémentation des compresseurs.....</b>               | <b>5</b>  |
| 2.1. BitPackingNoOverlap ('k' bits fixes sans chevauchement).....      | 5         |
| 2.2 BitPackingWithOverlap ('k' bits fixes avec chevauchement).....     | 6         |
| 2.3. BitPackingWithOverflow (Heuristique avec overflow).....           | 6         |
| <b>3. Protocole de Test et Analyse des Performances.....</b>           | <b>7</b>  |
| 3.1. Protocoles de Mesure et Outils.....                               | 7         |
| 3.1.1. Chargement des données.....                                     | 7         |
| 3.1.2. Mesure du temps d'exécution.....                                | 7         |
| 3.1.3. Validation de l'intégrité.....                                  | 8         |
| 3.1.4. Outils de Débogage.....                                         | 8         |
| 3.1.5. Test interactif du get().....                                   | 8         |
| 3.2. Analyse des Benchmarks : Qualités et Limites.....                 | 8         |
| 3.2.1. La limite du Gaspillage : de NoOverlap à WithOverlap.....       | 8         |
| 3.2.2. Le problème du 'k' unique : de WithOverlap à WithOverflow.....  | 9         |
| 3.2.3. La limite de l'heuristique : WithOverflow ou WithOverlap ?..... | 9         |
| <b>4. Analyse de rentabilité et Seuil de latence 't'.....</b>          | <b>10</b> |
| 4.1. Formulation de l'équation.....                                    | 10        |
| 4.2. Implémentation et Exemple de Résultat.....                        | 10        |
| <b>Conclusion.....</b>                                                 | <b>11</b> |

# Introduction

Le problème auquel on fait face est le suivant : le stockage de tableaux d'entiers utilisant une représentation standard de 32 bits par valeur est souvent inefficace, gaspillant un espace mémoire considérable.

L'objectif de ce projet est donc d'implémenter et d'évaluer différentes stratégies de compression pour réduire cette empreinte mémoire.

Pour répondre à cette problématique, le projet a été structuré autour des principes de programmation orientée objet vus en cours.

Une interface BitPacker définit le contrat commun (polymorphisme), tandis qu'une BitPackerFactory applique le 'factory pattern' pour instancier la stratégie de compression désirée, masquant ainsi les détails de l'implémentation.

Ce rapport analysera une progression de trois stratégies de compression, chacune répondant aux limitations de la précédente :

- 'k' bits fixe simple : Une approche utilisant un nombre 'k' de bits fixe mais qui génère une perte d'espace si 'k' n'est pas un diviseur de 32.
- 'k' bits fixe optimisé : Une version améliorée qui utilise 'k' bits fixes sans gaspillage, en permettant aux données de chevaucher les frontières des mots de 32 bits.
- Heuristique avec overflow : Une approche avancée qui abandonne le 'k' unique. Elle sépare les "petites valeurs" des "grandes valeurs", ces dernières étant déportées dans une zone dédiée.

Ce rapport sera donc structuré comme suit :

- D'abord, je présenterais les choix de conception POO (Interface et Factory) qui constituent l'architecture du projet.
- Ensuite, je détaillerais la logique d'implémentation de chacune des trois stratégies de compression.
- Puis, je détaillerai le protocole de test et de mesure des performances avant de mener une analyse de benchmarks ciblés afin de révéler les qualités et les limites de chaque compresseur.
- Enfin, je finirais sur une analyse de rentabilité afin de calculer le seuil de latence 't' théorique, déterminant à partir de quel moment le coût de la décompression est amorti par le gain en transmission.

## 1. Architecture et Choix de conception

Pour répondre aux exigences du projet tout en construisant un système propre et maintenable, une architecture flexible a été mise en place. L'organisation du code s'articule autour de trois

principes de conception majeurs : le polymorphisme, le pattern 'factory' et la séparation des responsabilités.

## 1.1. Le Polymorphisme (Interface BitPacker)

La classe BenchmarkRunner contient la logique de compression, son rôle est d'orchestrer l'ensemble du processus de test : charger les données, lancer la compression, la décompression, et l'accès aux éléments du tableau, tout en mesurant les performances de chaque opération.

Cependant, étant donné que je devais implémenter trois stratégies de compression distinctes utilisant ces mêmes fonctions, j'ai fait face à un problème de conception majeur.

En effet, écrire des if/else pour chaque appel de méthode dans le BenchmarkRunner créerait un mauvais code difficile à maintenir.

C'est pour cela que j'ai créé l'interface BitPacker qui définit le contrat public avec les méthodes compress, decompress, get que toutes les stratégies de compression vont implémenter par la suite.

Ici, le BenchmarkRunner ne dépend plus des compresseurs mais a recours seulement à l'interface directement.

Ainsi, ce choix de conception respecte deux principes vus en cours :

- Le principe d'inversion des dépendances ('D' de SOLID)
- Le principe Ouvert/Fermé ('O' de SOLID)

## 1.2. Le Pattern 'Factory'

Créer une factory pour gérer la création du type de compression était une étape importante dans le choix architectural du code, permettant ainsi de découpler le BenchmarkRunner de la logique d'instanciation des compresseurs.

Sans cette approche, je me serais retrouvé avec un enchaînement de if ou de switch dans le BenchmarkRunner afin de déterminer quelle compresseur instancier en fonction du paramètre choisi.

Pour éviter cet anti-pattern, j'ai implémenté la classe BitPackerFactory qui permet de centraliser cette logique de création dans une unique méthode statique create(CompressionType type).

Grâce à ça, le BenchmarkRunner n'a plus qu'à appeler BitPackerFactory.create(type) sans jamais connaître les classes concrètes qu'il reçoit en retour.

Cette approche est une application directe des 'Creational Patterns' vus en cours. En effet, mon implémentation est une forme de Factory Method, similaire à l'exemple FabriqueAnimal du

cours. La principale différence est que mon implémentation s'appuie sur un enum (CompressionType) plutôt qu'un String.

De plus, elle respecte le principe Ouvert/Fermé. En effet, on peut ajouter d'autres compresseurs en modifiant seulement la Factory sans jamais toucher au BenchmarkRunner.

### 1.3. La Séparation des Responsabilités (Classe Utilitaire)

La méthode de compression avec Overflow que je vais détailler par la suite a été la plus complexe à implémenter. En effet, elle doit gérer à la fois la logique de haut niveau (les pointeurs, l'heuristique, le layout mémoire) et les opérations de bas niveau (lire et écrire des bits spécifiques).

Le problème que j'ai rencontré ici est que si cette logique était concentrée dans une unique classe, cela créerait un 'Objet Divin' qui rendrait la classe très dense et difficile à gérer.

Pour éviter cet anti-pattern j'ai adopté une approche qui extrait toute la partie bas niveau dans une classe utilitaire statique 'CompressionUtils'. Cette classe contient les fonctions 'readBits', 'writeBits', 'bitsFor' et l'heuristique 'findBestKByBitJump' que je détaillerai par la suite.

Ce choix de conception a été fait de la sorte afin d'appliquer le principe de Responsabilité Unique ('S' de SOLID). En effet, ici la classe 'BitPackingWithOverflow' est responsable de l'orchestration de la compression avec overflow, tandis que 'CompressionUtils' est responsable de l'exécution des opérations de bits.

Ainsi, cette séparation rend le code global plus lisible et plus facile à tester et déboguer.

## 2. Logique d'implémentation des compresseurs

Comme présenté rapidement dans l'introduction, 3 stratégies de compression ont dû être implémentées afin de répondre à la problématique de l'énoncé. Toutes respectent l'interface BitPacker et la contrainte d'accès direct ce complexité  $O(1)$ .

### 2.1. BitPackingNoOverlap ('k' bits fixes sans chevauchement)

Cette classe implémente la première version expliquée en introduction.

- **compress(int[] tab)** : la fonction détermine d'abord le nombre de bits 'k' requis en trouvant la plus grande valeur du tableau. Elle calcule ensuite combien de valeurs de 'k' bits peuvent tenir dans un seul int de 32 bits et ce en utilisant une simple division entière, créant ainsi des zones fixes. Elle remplit un tableau de sortie, en plaçant chaque nouvelle valeur dans la zone suivante.

- **get (int i)** : Pour trouver la i-ème valeur, la fonction calcule dans quel int du tableau compressé elle se trouve et ce via une division entière  $i / \text{valuesPerInt}$  et à quelle position à l'intérieur de cet int et ce via  $i \% \text{valuesPerInt}$ .
- **decompress()** : elle va créer un nouveau tableau et va parcourir le tableau compressé et effectuer N fois get(i) afin de récupérer chaque valeur du tableau initial.

## 2.2 BitPackingWithOverlap ('k' bits fixes avec chevauchement)

Cette classe implémente la deuxième version qui peut écrire des entiers compressés sur deux entiers consécutifs.

- **compress(int[] tab)** : La fonction détermine 'k' de la même manière que la méthode précédente. Cependant, au lieu de créer des zones, elle va allouer un flux de bits continu de taille  $N * k$ . Les valeurs sont donc écrites les unes à la suite des autres en ignorant les frontières de 32 bits.
- **get (int i)** : La fonction calcule la position du bit de départ ( $i * k$ ) et lit les bits à partir de cette position. Si les 'k' bits sont à cheval sur deux ints ( $\text{bitsRead} < \text{bitsPerValue}$ ), la fonction lit la fin du premier int et le début du second puis les ré-assemble.
- **decompress()** : Pour des raisons de performances, cette fois au lieu d'appeler get() N fois, la fonction effectue une lecture séquentielle en maintenant un curseur de bits (`currentBitOffset`).

## 2.3. BitPackingWithOverflow (Heuristique avec overflow)

- **compress (int[] tab)** : Cette fois, au lieu d'un seul 'k' basé sur la valeur maximale, une heuristique 'findBestKByBitJump' va essayer de trouver un 'k\_normal' optimal. En effet, elle va analyser la distribution de toutes les tailles de bits et identifier le plus grand saut de bits. Le k\_normal est fixé à la valeur juste avant ce saut. De plus, une logique d'adaptation est appliquée : l'algorithme compare 'k\_normal' au 'k\_index' qui représente le nombre de bits requis pour stocker l'index des valeurs d'overflow. Si  $k\_index > k\_normal$ , le seuil de coupure est relevé et ainsi, cela permet de sauver des valeurs 'moyennes' de l'overflow, optimisant ainsi l'espace.

Ainsi, les valeurs  $\leq k\_normal$  sont considérés comme des petites valeurs et les autres des grandes valeurs qui vont être ajoutées à une liste d'overflow.

Le tableau compressé est alloué en un seul bloc avec deux sections, une grande section de N pointeurs au début et une zone d'overflow à la fin. L'écriture se fait en deux temps :

- le code parcourt le tableau d'origine, si la valeur est "petite", il l'écrit directement dans son emplacement de pointeur ([0][valeur])
- Si la valeur est "grande", il l'ajoute à la zone d'overflow et écrit le pointeur [1][index\_overflow] qui indique où la trouver.

- **get(int i)** : La fonction lit le i-ème pointeur et vérifie le bit de flag. Si le flag est 0, elle retourne le reste du pointeur, ce qui correspond à la petite valeur. Si le flag est 1, elle lit l'index d'overflow, calcule l'adresse complète dans la zone d'overflow en utilisant overflowDataStartBit et va y lire la valeur finale.
- **decompress()** : La fonction est une simple boucle qui appelle get(i) N fois (simple à implémenter).

### 3. Protocole de Test et Analyse des Performances

La section suivante s'articule en deux temps. Elle va présenter d'abord le protocole d'analyse des performances de chaque compresseur suivi des outils de débogage et d'affichage implémentés pour mieux tester notre programme.

Elle se concentrera ensuite sur l'analyse de benchmarks ciblés qui ont permis de révéler les qualités, mais aussi les limites de chaque compresseur.

#### 3.1. Protocoles de Mesure et Outils

Afin d'assurer des mesures fiables pour chaque méthode et de bien tester mon programme, un protocole bien défini a été implémenté et principalement géré par la classe BenchmarkRunner.

##### 3.1.1. Chargement des données

Afin de tester mon programme, les données sont lues depuis un fichier .txt via la classe Utils. Si un élément non-entier est détecté (float, lettre, symbole etc...), une exception est levée pour garantir que seules des données valides sont traitées.

##### 3.1.2. Mesure du temps d'exécution

Pour chronométrer les opérations compress, decompress et get, j'ai tout simplement utilisé System.nanoTime(), l'API Java pour le chronométrage. Les durées sont ensuite converties dans

un format lisible, avec l'unité la plus pertinente (ns,  $\mu$ s, ms, s) par la fonction `formatDuration()` de la classe `Utils`.

### **3.1.3. Validation de l'intégrité**

Une étape de validation intervient après chaque cycle de compression/décompression. En effet, le tableau décompressé est directement comparé bit à bit au tableau original via `Arrays.equals()`. Ainsi, le programme affiche un message clair de succès ou d'échec afin de garantir le bon fonctionnement de chaque méthode.

### **3.1.4. Outils de Débogage**

Une fonctionnalité clé du `BenchmarkRunner` que j'ai implémenté et qui a été très utile pour le débogage a été l'affichage binaire des données compressées.

Cela a été indispensable pour vérifier le bon alignement des bits et le bon fonctionnement de chaque compresseur, à savoir le gaspillage de bits dans la version sans chevauchement, le chevauchement dans la deuxième version et la structure `Pointeur/Overflow` dans la dernière version du compresseur.

### **3.1.5. Test interactif du `get()`**

Pour valider l'accès direct en  $O(1)$ , un mode interactif est lancé, permettant à l'utilisateur de demander la valeur à n'importe quel index. Le temps d'accès spécifique pour ce `get()` est alors mesuré et affiché.

## **3.2. Analyse des Benchmarks : Qualités et Limites**

Plusieurs fichiers de test ont été conçus pour pouvoir tester chaque compresseur. L'objectif étant d'identifier les scénarios où ils sont excellents, ainsi que les cas qui révèlent leurs limites.

### **3.2.1. La limite du Gaspillage : de `NoOverlap` à `WithOverlap`**

En testant quelques fichiers au début, on comprend vite que l'implémentation du premier compresseur est efficace dans les cas où 'k' est un diviseur de 32 (`no_overlap/parfait_8bit.txt` par exemple). Chaque entier de sortie stocke 4 valeurs sans perdre aucun bit.

Cependant, la faiblesse et les limites de ce compresseur sont vite aperçus avec le test `no_overlap/gaspillage.txt` par exemple. La présence d'une très grande valeur (65536) avec que des petites valeurs force 'k' à 17 bits.

Chaque entier de 32 bits ne stocke qu'une valeur de 17 bits, gaspillant ainsi 15 bits à chaque fois, rendant la compression totalement inutile.



C'est là que la version avec chevauchement intervient pour résoudre ce problème, réduisant ainsi la taille totale du tableau compressé.

### **3.2.2. Le problème du 'k' unique : de WithOverlap à WithOverflow**

Certes, le deuxième compresseur résout le problème de gaspillage rencontré précédemment mais est confronté à sa propre limite : 'le problème de la valeur maximale'.

En effet, le test `with_overlap/limite.txt` illustre parfaitement ce problème : une majorité de petites valeurs est pénalisée par une unique grande valeur. Ici, l'algorithme est forcé de choisir un 'k' de 22 bits pour toutes les valeurs, ce qui est extrêmement inefficace.

C'est là que la version avec overflow intervient pour résoudre ce problème.

### **3.2.3. La limite de l'heuristique : WithOverflow ou WithOverlap ?**

Confronté au même fichier que le précédent, cette fois l'heuristique du troisième compresseur détecte le saut énorme de 3 à 22 bits.

Ainsi, elle fixe le `k_normal` à 3, allouant un pointeur de 4 bits pour chaque élément (1 bit de flag et 3 bits de payload). Il déporte l'unique grande valeur en overflow, faisant passer la taille du tableau compressé de 6 entiers à 2 entiers seulement.

Cependant, cette heuristique possède deux limites importantes où elle devient contre-productive.

La première limite est illustrée par le test `with_overflow/limite.txt`. Ici, on se retrouve dans un cas où le saut entre les tailles de bits est toujours de 1.

Sachant que l'heuristique considère un saut significatif seulement si il est  $> 1$ , elle va donc fixer le 'k\_normal' à la valeur la plus grande (17 dans notre exemple).

Donc, aucune valeur n'est placée en overflow et chaque valeur va utiliser un bit de flag en plus des 'k\_normal' bits.

Cette limite de l'heuristique rend l'utilisation du précédent compresseur plus pratique dans ce genre de cas.

La seconde limite est illustrée par les tests `with_overflow/limite2.txt` et `with_overflow/limite3.txt`.

Dans le premier test, le saut est  $> 1$  et toujours uniforme, on se retrouve dans un cas où l'heuristique s'arrête au premier saut et ignore les autres sauts car ils sont égaux. Elle fixe 'k\_normal' à la valeur juste avant qui est de 1 bit.

Dans le deuxième test, l'heuristique s'arrête au plus grand saut qui est le premier et ignore le reste. Elle fixe 'k\_normal' à la valeur juste avant qui est aussi de 1 bit.

Ainsi, l'algorithme déporte la quasi-totalité des valeurs en overflow. En effet, ici le compresseur devient contre-productif puisqu'il doit stocker N pointeurs en plus de N-1 valeurs dans la zone d'overflow.

Au final, à cause du coût de gestion des pointeurs, l'algorithme se retrouve à consommer plus d'espace que le précédent compresseur.

## 4. Analyse de rentabilité et Seuil de latence 't'

On a pu voir que la compression a permis de réduire l'espace, cependant son principal rôle est surtout d'accélérer la transmission.

Je vais donc analyser la rentabilité qui représente une latence 't' à partir de laquelle le temps gagné en transmission dépasse le temps perdu dans la compression et décompression.

Afin d'établir l'équation, on définit les variables suivantes :

- N : le nombre d'entier dans le fichier source
- N\_comp : le nombre d'entier dans le tableau compressé
- T\_comp : temps de calcul pour compress()
- T\_decomp : temps de calcul pour decompress()
- t : temps de latence

### 4.1. Formulation de l'équation

- T\_classique (temps sans compression) :  $N * t$
- T\_avec\_compression :  $T_{comp} + T_{decomp} + (N_{comp} * t)$

La compression devient rentable lorsque  $T_{avec\_compression} < T_{classique}$

Lorsqu'on remplace avec les variables cités on obtient :

- $T_{comp} + T_{decomp} + (N_{comp} * t) < N * t$   
 $= T_{comp} + T_{decomp} < (N * t) - (N_{comp} * t)$   
 $= T_{comp} + T_{decomp} < t * (N - N_{comp})$   
 $= t > (T_{comp} + T_{decomp}) / (N - N_{comp})$

### 4.2. Implémentation et Exemple de Résultat

L'analyse de rentabilité a été directement implémentée dans la classe BenchmarkRunner. Cela m'a permis d'appliquer la formule sur n'importe quel benchmark. Après validation de l'intégrité des données, le programme calcule et affiche ce seuil.

On prend par exemple le test with\_overflow/ideal2.txt exécuté avec la compression avec overflow.

Les mesures qu'on obtient sont :

- $N = 28$
- $N_{\text{comp}} = 6$
- $T_{\text{comp}} = 1,81 \text{ ms}$
- $T_{\text{decomp}} = 211,79 \mu\text{s}$

Afin d'obtenir le seuil, on fait  $t > (1,81 \text{ ms} + 211,79 \mu\text{s}) / (28 - 6)$ , ce qui nous donne  $t > 92,07 \mu\text{s}$ .

On en déduit que pour ce scénario, la compression est rentable si le temps de transmission pour un seul entier est supérieur à  $92,07 \mu\text{s}$ .

## Conclusion

Ce projet a débuté par un défi d'optimisation : réduire l'empreinte mémoire des tableaux d'entiers pour accélérer leur transmission.

La construction d'une architecture logicielle flexible, basée sur les principes de conception objet a permis d'implémenter et de comparer trois stratégies de compression distinctes.

L'implémentation et l'analyse de ces stratégies a pu démontrer qu'il n'existe pas réellement de compresseur parfait. Le choix optimal est plutôt un compromis qui dépend du contexte statistique des données.

On a pu observer une progression où chaque implémentation résolvait les défauts de la précédente, cependant l'analyse a prouvé que même le compresseur le plus optimal pouvait devenir contre-productif face à certaines distributions de données comme les progressions linéaires ou géométriques.

Le véritable enjeu n'est donc pas de trouver un compresseur universel, mais de savoir quand l'utiliser.

C'est l'aboutissement de ce projet : l'implémentation de l'analyse de rentabilité fournit la formule concrète du seuil 't'. Ce seuil quantifie l'arbitrage entre le coût en calcul et le gain en transmission, offrant ainsi un critère de décision objectif pour justifier (ou non) la compression.